

Devoir surveillé

Durée 1h30

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.*

Exercice 1 (Cours - 4 pts)

1°) Donnez la représentation mémoire d'un processus en précisant chaque zone avec son rôle.

2°) Qu'est-ce qu'un inode ?

3°) Qu'est-ce que l'attente active et l'attente passive ? Donnez des exemples.

Exercice 2 (Surveillance de processus - 5 pts)

Nous souhaitons mettre en place un système de surveillance de processus. Il est basé sur deux programmes : le *launcher* et le *watcher*. Le *launcher* démarre un programme et enregistre le processus correspondant auprès du *watcher*. Ce dernier attend l'enregistrement de processus et en même temps, il vérifie périodiquement que les processus enregistrés sont en cours d'exécution. Il affiche un message lorsqu'un processus s'est arrêté. L'enregistrement est réalisé à l'aide de signaux.

1°) Comment le *launcher* peut-il connaître le PID du processus démarré et comment peut-il l'envoyer au *watcher* ?

2°) Expliquez le fonctionnement complet du *launcher* et précisez tous les appels systèmes nécessaires. Il prend en arguments le nom du programme à exécuter et le PID du *watcher*.

Nous supposons que le *watcher* maintient la liste des processus à surveiller sous la forme d'un tableau.

3°) Comment le *watcher* peut-il vérifier que les processus surveillés sont toujours en cours d'exécution ? Précisez les appels systèmes.

4°) Donnez la partie du code permettant d'attendre le signal provenant du *launcher*. Cette attente doit être limitée à 0.5s.

Exercice 3 (Traitements par flot - 6 pts)

Nous souhaitons mettre en place un flot de traitements sur des données réalisés par différents programmes indépendants (nommés *calculateur*). Chacun prend une série de données (pour simplifier, des entiers) saisie au clavier et écrit le résultat (une suite d'entiers) dans le terminal. Le résultat d'un *calculateur* peut être utilisé comme entrée pour un autre *calculateur*.

1°) Avec 3 *calculateurs* nommés prog1, prog2 et prog3, donnez la ligne de commande dans le terminal pour réaliser le flot de traitement.

2°) Comment peut-on effectuer une redirection en C ? Précisez tous les appels systèmes.

Pour exécuter les *calculateurs*, nous écrivons un programme qui prend comme arguments les noms des programmes à exécuter. Il les exécute ensuite en redirigeant les entrées/sorties.

3°) Représentez l'architecture de l'application. Précisez les entrées/sorties.

4°) Expliquez les différentes actions à réaliser pour ce programme. Précisez tous les appels systèmes.

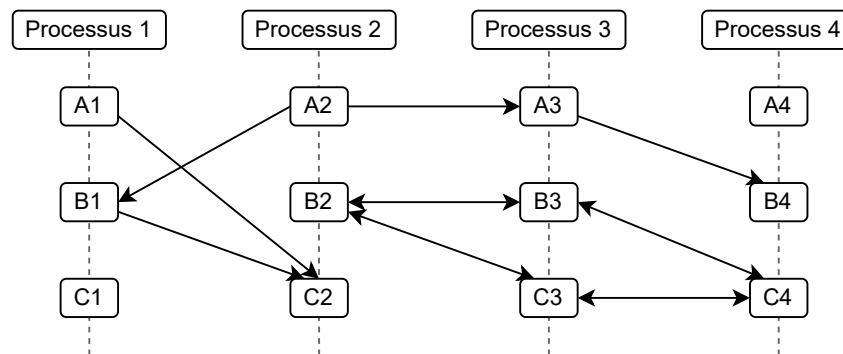
Nous souhaitons écrire la fonction `child(int in, int out, char *program)` où `in` est le descripteur de fichier correspondant à l'entrée du programme, `out` la sortie et `program` le nom du programme à exécuter.

5°) Expliquez pour chaque fils les valeurs à passer pour `in` et `out`.

6°) Écrivez la fonction `child`.

Exercice 4 (Sémaphores de Dijkstra et graphe d'allocation de ressources - 5 pts)

Soit le diagramme de précédence suivant, représentant 4 processus contenant chacun 3 blocs s'exécutant les uns après les autres :



1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

2°) Proposez une solution pour réduire le nombre de sémaphores tout en conservant les contraintes du diagramme.

Nous supposons une ressource R_1 comportant 2 exemplaires et 3 processus P_1 , P_2 et P_3 . Les trois processus demandent simultanément l'accès à un exemplaire de R_1 .

3°) Donnez toutes les étapes de l'exécution des demandes jusqu'à ce que tous les processus relâchent la ressource. Utilisez des graphes d'allocation de ressources différents.

4°) D'une manière générale, est-il possible d'avoir un cas d'interblocage avec une ressource possédant deux exemplaires ?

Annexes : quelques en-têtes de fonctions C

```

void          _exit(int code);
unsigned int  alarm(unsigned int nb_secondes);
int           atexit(void (*procedure)(void));
void          clearerr(FILE *flux);
int           close(int fd);
int           closedir(DIR *repertoire);
int           creat(const char *chemin, mode_t mode);
int           dup(int ancien_fd);
int           dup2(int ancien_fd, nouveau_fd);
int           execve(const char *fichier, char *const argv[], char *const envp[]);
void          exit(int statut);
int           fcntl(int fd, int commande, ...);
int           fclose(FILE *flux);
FILE          *fdopen(int fd, const char *mode);
int           feof(FILE *flux);
int           ferror(FILE *flux);
int           fflush(FILE *flux);
int           fileno(FILE *flux);
FILE          *fopen(const char *chemin, const char *mode);
pid_t         fork();
size_t        fread(void *tampon, size_t taille, size_t nb, FILE *flux);
FILE          *freopen(const char *chemin, const char *mode, FILE *flux);
int           fseek(FILE *flux, long offset, int point_depart);
int           fstat(int fd, struct stat *tampon);
int           ftell(FILE *flux);
size_t        fwrite(const void *tampon, size_t taille, size_t nb, FILE *flux);
pid_t         getpid();
pid_t         getppid();
int           kill(pid_t pid, int numero_signal);
off_t         lseek(int fd, off_t offset, int point_depart);
int           lstat(const char *chemin, struct stat *tampon);
void          *memcpy(void *destination, const void *source, size_t taille);
void          *memset(void *tampon, int caractere, size_t nombre_elements);
int           mkfifo(const char *nomFichier, mode_t mode);
int           open(const char *chemin, int options);
int           open(const char *chemin, int options, mode_t mode);
DIR           *opendir(const char *chemin);
int           pause(void);
int           pipe(int tube[2]);
ssize_t       read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void          rewind(FILE *flux);
void          rewinddir(DIR *repertoire);
void          (*signal(int numero_signal, void (*handler)(int)))(int)
int           sigaction(int numero_signal, const struct sigaction *action,
                        struct sigaction *ancienne_action);
int           sigaddset(sigset_t *ensemble, int numero_signal);
int           sigdelset(sigset_t *ensemble, int numero_signal);
int           sigemptyset(sigset_t *ensemble);
int           sigfillset(sigset_t *ensemble);
int           sigismember(sigset_t *ensemble, int numero_signal);
int           sigpending(sigset_t *ensemble);
int           sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int           sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int           sigsuspend(const sigset_t *ensemble);
int           sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int           sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                           const struct timespec *timeout);
unsigned int  sleep(unsigned int nombre_secondes);
int           stat(const char *chemin, struct stat *tampon);

```

```

time_t      time(time_t *temps);
int          truncate(const char *chemin, off_t offset);
int          ftruncate(int fd, off_t offset);
int          truncate64(const char *chemin, off64_t offset);
int          ftruncate64(int fd, off64_t offset);
int          unlink(const char *nomFichier);
pid_t        wait(int *statut);
pid_t        waitpid(pid_t pid, int *statut, int options);
ssize_t      write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_départ (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED
valeurs de errno : E2BIG, EACCESS, EAGAIN, EBADE, EBADF, EBADFD, EBADMSG, EBADR, EBADRQC, EBADSLT, EBUSY, ECANCELED, ECHILD, ECHRNG, EDEADLK, EDOM, EDQUOT, EEXIST, EFAULT, EFBIG, EHOSTDOWN, EIDRM, EILSEQ, EINPROGRESS, EINTR, EINVAL, EIO, EISDIR, EISNAM, ELOOP, EMEDIUMTYPE, EMFILE, EMLINK, EMSGSIZE, EMULTIHOP, ENAMETOOLONG, ENFILE, ENOBUFS, ENODATA, ENODEV, ENOENT, ENOEXEC, ENOKEY, ENOLCK, ENOLINK, ENOMEDIUM, ENOMEM, ENOMSG, ENOSPC, ENOSR, ENOSTR, ENOSYS, ENOTBLK, ENOTCONN, ENOTDIR, ENOTEMPTY, ENOTSUP, ENOTTY, EOVERFLOW, EPERM, EPFNOSUPPORT, EPIPE, ERANGE, ERESTART, EROFS, ESPIPE, ESRCH, ESTALE, ESTRPIPE, ETIME, ETXTBSY, EUCLEAN, EXDEV, EXFULL

Annexes : macros

Sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG
 Sur le champ **st_mode** de struct stat : S_ISREG, S_ISDIR, S_ISFIFO

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int      sa_flags;
};

union sigval_t {
    int      sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int      si_signo;
    int      si_code;
    sigval_t si_value;
    pid_t    si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```